Journal Title
XX(X):1–14

DOI: 10.1177/ToBeAssigned
www.sagepub.com/
SAGE

Streaming Latency Benchmarks for Real-Time Football Feeds: REDIS Streams versus Apache KAFKA, and the Physical-Consistency Gate That Invalidated Our First Answer

Gustavo Pedro Ricou[1]

Abstract

Live football analytics runs on streaming middleware, and the choice between Apache KAFKA and REDIS Streams is usually made on grey-literature benchmarks conducted at arrival rates no sport produces. We benchmark the two on the StatsBomb open dataset (2003–2023; 52 competition-seasons, 3,315 matches), driving each broker with real match feeds replayed at their true event rate, at concurrency levels derived from real kick-off schedules rather than chosen by hand, with open load-generation code.

Our first campaign produced a complete and theory-confirming answer: REDIS transport rising monotonically with concurrency while KAFKA stayed flat, $p = 6.7 \times 10^{-12}$, complete rank separation, exactly as a single-threaded server should behave. It was wrong. Auditing every run against a physical constraint — a broker cannot acknowledge a message after the consumer has logged receiving it — condemned 862 of $1{,}382$ single-host runs and 459 of 884 multi-host runs. The inversions are invisible in every summary statistic: medians stay plausible, confidence intervals stay tight, and the effect agrees with theory. We report this as the study's primary finding, define the gate as a stated rule (`clock_integrity.py`), and re-run the study inside it.

What survives answers the original question, with a different emphasis than we expected. On *end-to-end* lag REDIS Streams is decisively better: median 5.2 ms against KAFKA's 105.5 ms, at every concurrency level football produces ($N \leq 12$), a factor of 20. On *broker transport* the two are statistically equivalent within 1 ms (0.84 vs 0.80 ms) and neither degrades with concurrency. The gap is therefore not a broker property but a client one: it lives entirely in the producer send path, and it appears only at football's sparse arrival rate — under $10\times$-accelerated replay it vanishes, which is precisely why conventional benchmarks miss it. The ordering reverses under network delay, where KAFKA tracks the injected latency (78 ms at 20 ms delay) while REDIS collapses to 31 seconds through a round-trip-bound acknowledgement pattern; batching the acknowledgements recovers a factor of 40.2 at realistic load, and unexplainedly fails to at $5\times$ that load, which we report as an open question. Each system has exactly one client-side setting that moves latency by one to two orders of magnitude, and neither is the documented default.

Keywords

Real-time sports analytics, football, streaming systems, Apache Kafka, Redis Streams, latency benchmarking, measurement validity, reproducible research, Age of Information

1 Introduction

A live football analytics pipeline — an in-play odds engine, a coaching dashboard, a broadcast statistics overlay — moves event data from an annotation source to a model through streaming middleware. Two products dominate that role. Apache KAFKA Kreps et al. (2011) is a partitioned, replicated distributed commit log built for durable high-throughput pipelines. REDIS Streams Sanfilipe (2017) is an in-memory append-only structure with consumer groups, built for minimal per-operation delay. Practitioners choose between them largely on grey-literature benchmarks that report REDIS as two to five times faster Diaries (2025); Yerrapragada (2025); JusDB (2025), none of which is conducted on a sports workload and most of which is not reproducible.

This paper asks the practical question directly. Using the StatsBomb open dataset StatsBomb (2023) across 2003–2023 and open load-generation scripts, we compare end-to-end lag for the two backends under the concurrency a football feed actually produces, and we ask where any difference comes from, under what deployment conditions it reverses, and what it costs to configure each system correctly.

[1]School of Computer Science and Statistics, Trinity College Dublin, Dublin, Ireland

Corresponding author:
Gustavo Pedro Ricou, School of Computer Science and Statistics, Trinity College Dublin, Dublin, Ireland.
Email: pedrorig@tcd.ie

The result we obtained first, and why this paper is not that paper. Our initial campaign returned a clean answer. REDIS broker transport rose monotonically with concurrency ($1.006 \rightarrow 1.197 \rightarrow 1.346$ ms across $N \in \{1, 5, 10\}$, Mann–Whitney $p = 6.7 \times 10^{-12}$ after Holm correction, complete rank separation at $N \geq 5$) while KAFKA stayed flat within 0.3%. The result was internally consistent across 1,382 runs, survived a fairness audit of payload sizes and client configuration, and matched the architectural prediction that a single-threaded server serialises concurrent streams. It would have been the paper.

It was an artefact. Broker transport is computed as consumer receipt minus broker acknowledgement, two timestamps taken in two processes. A message cannot be received before it is acknowledged, so a negative transport value is not noise but proof that the instrument has failed. Auditing every run against that constraint — rather than only the runs whose results looked implausible — condemned 862 of 1,382 single-host runs and 459 of 884 multi-host runs, including every condition behind the finding above.

The failure mode is the reason we make it the centre of the paper rather than a footnote. Timestamp inversion under load is invisible to every check a referee would apply. The medians stay positive and physically plausible; the confidence intervals stay tight; the effect sizes are large; the direction agrees with theory. Only about 5 to 14% of individual events invert, which is enough to bias a median by a fraction of a millisecond in a measurement whose whole effect is a fraction of a millisecond, and not enough to disturb any aggregate a reader would inspect. A benchmark can therefore produce a complete, coherent, theoretically congenial and entirely false result set, and no amount of statistical care will reveal it. A check against physical constraints will, and it has to be applied to every condition by rule.

Contributions.

1. **A clock-integrity gate for cross-process latency benchmarks**, stated as a rule rather than a judgement, applied uniformly to all 2,266 runs in this study, released as under two hundred lines of open code. We report what it condemns, why the condemned data looked correct, and the general property that makes such benchmarks fragile: the gate binds hardest exactly where the measured effect is smallest (Section 6).
2. **The benchmark itself, inside the gate.** At the concurrency levels football produces, REDIS Streams delivers a median end-to-end lag of 5.2 ms against KAFKA's 105.5 ms, while the two brokers' own transport is equivalent within 1 ms and neither degrades with concurrency (Section 7.1).
3. **An attribution and a configuration account.** The $20\times$ end-to-end gap is a property of the client send path at football's sparse arrival rate, not of either broker, and it disappears under accelerated replay. Each system has exactly one client-side setting that moves latency by one to two orders of magnitude — KAFKA's `max.in.flight`, $103\times$; REDIS's acknowledgement batching, $40\times$ — and neither is the value the documentation's introductory example uses (Sections 7.3 and 8.2).
4. **A workload characterisation of football event feeds** across 3,315 matches, which supplies the arrival rate, burst structure and concurrency levels the benchmark is driven at, and which we believe has not been published before (Section 3).

Structure. Section 2 positions the work against the streaming benchmark literature. Section 3 characterises the workload and derives the experimental parameters from it. Section 4 describes the two testbeds, the metrics, the gate and the statistics. Section 5 presents the first result set in full; Section 6 audits it; Section 7 reports what survives. Section 8 answers the practical question and states what remains open.

1.1 Research Questions

RQ1 (end-to-end lag): Replaying real match feeds at their true event rate, what end-to-end lag do Apache KAFKA and REDIS Streams deliver, and do they differ?

RQ2 (concurrency): Does that lag change across the concurrency levels football actually produces, and does either backend degrade faster than the other?

RQ3 (attribution): Where in the pipeline does any difference live — in broker transport, or in the client path either side of it?

RQ4 (deployment conditions): Does the answer survive a network hop between consumer and broker, and what mechanism governs the failure when it does not?

RQ5 (measurement validity): How much of a conventionally conducted benchmark of this kind survives an explicit physical-consistency check, and what does the condemned portion look like beforehand?

RQ5 is not a robustness appendix. It is answered first, in Sections 5–6, because its answer determines which evidence RQ1–RQ4 may draw on.

1.2 Sports latency requirements

Real-time sports analytics imposes latency constraints that vary by stakeholder (Table 1). Betting platforms require sub-500 ms updates for in-play odds Solutions (2025); Ververica (2025); broadcast overlays tolerate up to a few seconds OptiView (2025); Promwad (2025); coaching decision support is quoted at under 500 ms Pappas et al. (2020); Sports (2023). These budgets are *asserted* by practitioners rather than derived from any measurement of what missing them costs, and we use them as the practitioner context our results are interpreted against, not as a success criterion. One consequence is worth stating at the outset: every configuration we measure in Section 7 meets every budget in the table, and the interesting differences between the two systems are two orders of magnitude below the tightest of them — until a network hop is introduced, at which point one configuration misses every budget by four orders of magnitude.

Table 1. Sports analytics latency requirements by use case and stakeholder, as asserted in the practitioner literature. These are stated budgets, not measured costs of missing them.

Use Case	Stakeholder	Latency Range	Critical Threshold
Betting Platforms			
Live Odds Update	Betting Platform	100–500ms	$<$ **500ms**
In-Play Betting	Betting Platform	50–200ms	$<$ **200ms**
Broadcasting			
Live Video Sync	Broadcaster	500–3000ms	$<$ **3000ms**
Live Stats Overlay	Broadcaster	100–1000ms	$<$ **1000ms**
Interactive Features	Broadcaster	200–500ms	$<$ **500ms**
Coaching			
Tactical Adjustment	Coach	200–800ms	$<$ **800ms**
Player Substitution	Coach	500–1500ms	$<$ **1500ms**
Fan Applications			
Push Notifications	Fan	1000–3000ms	$<$ **3000ms**
Live Updates	Fan	500–2000ms	$<$ **2000ms**
Post-Match Analysis			
Initial Analysis	Analyst	5000–10000ms	$<$ **10000ms**

2 Related Work

Streaming platforms. Stonebraker et al. Stonebraker et al. (2005) set out eight requirements for real-time stream processing, among them keeping data moving and processing with predictably low latency. KAFKA Kreps et al. (2011) realises them as a partitioned, replicated commit log: throughput through batching and sequential disk writes, parallelism through partitions. REDIS Streams Sanfilipe (2017) takes the opposite position, an in-memory append-only structure trading capacity for minimal per-operation delay. The architectural consequence matters here: KAFKA's partitioning admits genuine intra-broker parallelism, whereas a REDIS node executes commands on a single thread, so concurrent streams contend for one execution context. That prediction is what our first, invalid result set appeared to confirm. The CAP theorem Brewer (2012) and PACELC Abadi et al. (2012) frame the durability–latency trade-off both systems expose.

How streaming systems are benchmarked, and how it goes wrong. Evaluations of KAFKA are numerous He et al. (2020); Gai and Qiu (2020), as are cross-framework comparisons Carbone et al. (2015). Several works compare KAFKA against REDIS Streams directly Pandey et al. (2021); Zhang and Lee (2022). A recent survey of benchmark practice in KAFKA event-streaming systems Anonymous (2025) observes that reported results are frequently not comparable, because client configuration, acknowledgement semantics and measurement instrumentation differ between the systems under test. This study is a strong instance of that critique, and extends it: we find that comparability of *configuration* is not sufficient, because the instrumentation itself can fail silently under load. Zhang et al. Zhang et al. (2021) propose time-to-insight as an evaluation metric; we adopt it but decompose it, which is what makes the failure visible at all.

Established benchmarks, and why none answers this question. Linear Road Arasu et al. (2004) is the canonical streaming application benchmark, posing toll and traffic queries over a simulated vehicle stream; NEXMark Tucker et al. (2008) models an auction site and has become the standard query suite for dataflow engines; ESPBench Hesse et al. (2021) constructs an enterprise workload spanning transactional and analytical operators; RIoTBench Shukla et al. (2017) supplies IoT dataflows with realistic sensor arrival patterns; and the OpenMessaging Benchmark OpenMessaging Project, Linux Foundation (2018) is the closest analogue to this work, exercising broker throughput and latency directly across messaging systems.

Three things follow. First, each drives its system with a *synthetic* arrival process chosen for analytical tractability rather than fidelity to a domain. That is correct for a general-purpose benchmark and wrong for the question asked here; Section 3 derives the arrival process from the sport instead. This is not a stylistic preference: our central attribution result (Section 7.1) is that the difference between the two backends appears at football's sparse arrival rate and *vanishes* when the same corpus is replayed ten times faster. A fixed-rate synthetic publisher would have reported no difference. Second, these benchmarks are built to find a distributed system's breaking point, and football does not approach one. Third, none reports a physical-consistency audit of its own timestamps. We are not aware of a streaming benchmark that publishes what fraction of its runs it discarded on integrity grounds, which given our findings we take to be a gap in reporting practice rather than evidence that the problem is rare.

Real-time sports data systems. Sports analytics has moved from retrospective summary to live decision support Wright et al. (2022), driven by richer capture Link et al. (2021) and by open event data StatsBomb (2023). Architectural accounts of live football pipelines Pappas et al. (2020); Sports (2023) specify latency budgets for coaching and broadcast, and the betting industry is explicit that in-play latency is commercially decisive Solutions (2025); Ververica (2025). These sources establish that latency matters; they

assert budgets rather than measuring the transport term inside them.

Staleness and decision cost. The Age-of-Information literature formalises the cost of stale data: a monitor acting on information of age L incurs an error growing with L Kaul et al. (2012). For soccer, the state of the art in in-play win probability is the Bayesian model of Robberechts et al. Robberechts et al. (2021), calibrated on large event corpora and, like all such models, built under the assumption that event data arrives instantly. We use this framing lightly, in Section 7.6, to express our measured milliseconds on a decision-relevant scale — and we are explicit that it converts units rather than adding inferential power.

3 What a Football Event Feed Demands

The parameters of a latency benchmark — arrival rate, burst structure, number of concurrent feeds — are usually chosen for convenience. Because our central attribution result turns on the arrival rate being football's rather than a generator's, we derive all three from the corpus first. We use the StatsBomb open dataset across 2003–2023: 52 competition-seasons, 15 competitions, 3,315 matches, of which 2,806 are men's and 509 women's, fetched from a pinned upstream commit by `scripts/fetch_statsbomb_corpus.py`.

3.1 Arrival rate and burstiness

Football event data is sparse. A match yields a median of 3,507 events over roughly 8,412 seconds of match clock, a mean arrival rate of **0.415 events per second**. That is the figure a capacity calculation would use, and it understates the requirement: measured over a sliding ten-second window the same feeds peak at **2.70 events per second**, a peak-to-mean ratio of **6.45** (Table 2). Half of all inter-arrival gaps are at or below one second and the fifth percentile is indistinguishable from zero. Events cluster because football clusters: a restart, a sequence of quick passes, a goal and its surrounding play emit in tight groups separated by long quiet periods.

The structure is remarkably stable across the corpus. Every competition sits between 0.35 and 0.44 events per second with a peak-to-mean ratio between 6.2 and 7.9, and the men's and women's game differ by less than seven percent on both (0.419 vs 0.392 events per second; ratio 6.43 vs 6.61). The sparsest competitions are the burstiest — the African Cup of Nations and Indian Super League have the lowest mean rates and the highest ratios — so a low average is not licence to provision for the average.

Two consequences carry into the rest of the paper. A buffer sized from the mean is undersized more than six-fold. And, more importantly for what follows, an idle-to-busy duty cycle this extreme is exactly the regime in which a client that must wake from idle pays for doing so on every burst; Section 7.1 measures a 103 ms cost of precisely that shape.

3.2 How many feeds run at once

Every concurrency level in this literature is chosen by hand; our own earlier design swept $N \in \{1, 5, 10, 20\}$ and later $N \in \{10, 25, 50, 100\}$, which makes "does latency scale with concurrency?" a question about an invented variable. Football supplies the variable directly, because match records carry a kick-off date and time.

Across 2,346 distinct kick-off instants in the corpus, the largest carries **12 simultaneous matches**, and allowing for a nominal 115-minute duration the peak number of matches *in play* at any instant is **21**. The median slot carries one match. The maxima are not sampling accidents: domestic leagues schedule the final matchday simultaneously to protect competitive integrity, so a 20-team league plays ten concurrent matches by rule, and our largest observed slots are exactly such fixtures. We therefore benchmark at $N \in \{1, 9, 10, 12\}$: the median slot, the upper quartile of occupancy, the structural league-matchday bound, and the largest simultaneity observed.

Two caveats bound this, and we restate them wherever the numbers are used. StatsBomb open data is a *sample* of matches, not a complete record of any league-season, so observed simultaneity is a **lower bound**; the structural bound — half the teams in a league — is the safer planning figure. And the bound is *per league*. A feed platform serving many leagues and tiers at once faces their sum, which on a European Saturday afternoon runs to hundreds of matches. Our concurrency result should therefore be read as characterising a single-competition consumer, and the connection sweep of Section 7.4 is the arm that speaks, imperfectly, to the multi-league case.

4 Method

4.1 Two testbeds, both disclosed

Several of our earlier numbers turned out to be properties of a testbed rather than of either broker, so we state both fully and measure the parts that bound resolution (`scripts/platform_probe.py`; raw output in `docs/results/platform/`).

Testbed A (single host). Windows 11 (build 10.0.26200), 8-core AMD Ryzen, 31 GB RAM, CPython 3.9.13. Producer and consumer are native Windows processes; KAFKA 4.1.1 (KRaft) and REDIS 7.2.4 run as Docker containers under Docker Desktop, whose engine executes in a WSL2 virtual machine, reached through published ports. Two measured floors bound everything measured here: a requested 1 ms sleep takes a median of **15.6 ms** (the Windows timer tick), so event emission is quantised to a ≈15.6 ms grid; and establishing a TCP connection to a published container port costs a median 5.6 ms (REDIS) and 9.9 ms (KAFKA), far above a true loopback path, consistent with the ≈1 ms floor under which transport never falls.

Testbed B (multi-host). Four Oracle Cloud `VM.Standard.E5.Flex` virtual machines in `uk-london-1` — three brokers, one driver — on Ubuntu 22.04 (kernel 6.8.0-oracle), CPython 3.10.12, communicating over the tenancy's private network. Client library versions are pinned identically to Testbed A. Both platform floors fall away: a requested 1 ms sleep takes 1.06 ms, and an established-connection round trip to the broker is 0.22 ms on the same host and 0.24 ms across VMs (implying a ceiling of ≈4,200 request/response exchanges per second, against a demand of well under one). A genuine

Table 2. Football event-feed arrival statistics, medians across matches, from 3,315 StatsBomb matches spanning 2003–2023. Peak rate is the maximum over any sliding ten-second window. The peak-to-mean ratio is the quantity that sizes a buffer, and it is more than six times the figure a mean-rate calculation gives.

Scope	Matches	Mean rate (ev/s)	Peak rate (ev/s)	Peak/mean
All competitions	3,315	0.415	2.70	6.45
La Liga	867	0.434	2.70	6.31
Ligue 1	435	0.430	2.70	6.32
Premier League	418	0.407	2.60	6.50
Serie A	380	0.423	2.70	6.35
1. Bundesliga	340	0.423	2.70	6.42
FA Women's Super League	326	0.397	2.60	6.55
FIFA World Cup	128	0.404	2.65	6.61
Women's World Cup	116	0.386	2.60	6.82
Indian Super League	115	0.346	2.60	7.64
African Cup of Nations	52	0.342	2.55	7.89
Men's football	2,806	0.419	2.70	6.43
Women's football	509	0.392	2.60	6.61

two-machine network is thus *faster* than Testbed A's "co-located" path, which is the sharpest available statement of how much a testbed can distort a benchmark.

All results reported as findings in Section 7 come from Testbed B. Testbed A appears in Section 5 because it produced the result set this paper is about, and its floors are part of why.

4.2 Replay and load generation

Each match is preprocessed into a CSV *replay plan* recording, per event, its scheduled emission offset (`scripts/make_replay_plan.py`). A producer replays a plan at a configurable speed-up factor; a consumer reads events and records receipt and output times. In the concurrency sweep each of the N feeds carries a *distinct* real match at its true event rate, which matters: an earlier design in which every feed replayed the same merged ten-match plan made the concurrency axis covertly a throughput axis, raising the aggregate event rate roughly tenfold per step (Section 5.2).

Replay speed is itself a treatment in this study rather than a convenience. Headline results are at true real-time ($1\times$), which is both the ecologically correct choice and, as Section 6 shows, the only regime in which the instrument is sound. Where we report $10\times$ results we say so and give their gate status.

4.3 Metrics

The primary metric is Time-to-Insight (TTI), the interval from an event's *scheduled* emission to its availability at the consumer — the end-to-end lag of the paper's title. We decompose it as

$$\mathrm{TTI} = \underbrace{(t_{\mathrm{send}} - t_{\mathrm{sched}})}_{\text{producer scheduling lag}} + \underbrace{(t_{\mathrm{recv}} - t_{\mathrm{ack}})}_{\text{broker transport}} + (\text{residual}),$$

where t_{ack} is the broker acknowledgement, stamped by the KAFKA producer from its send callback and by the REDIS producer on return from `XADD`. The decomposition is not cosmetic and it is the reason this study reaches a different conclusion from its own first draft: TTI answers the deployment question (what staleness does a consumer experience?) while transport answers the architecture question (which broker is faster?), and the two have opposite answers here. We report both throughout and say which question each addresses. We further report p95, throughput, payload size and (de)serialisation overhead.

4.4 The clock-integrity gate

Because transport subtracts a timestamp taken in one process from a timestamp taken in another, it admits a check no statistic can supply: the result cannot be negative. A message cannot arrive before it is sent.

We implement this as a stated rule in `scripts/clock_integrity.py`, fixed before the final campaign and applied uniformly to every run in the study, including runs whose results looked entirely reasonable. A run is **condemned** if either

1. more than 1% of its events carry a negative transport value, or
2. the median of any latency component is negative.

A condition is reported as *usable* only if every one of its runs survives; where a condition is partially condemned we report the surviving runs and state the retention rate, and where the retention is low enough to constitute selection we say so. The tool exits non-zero when any run fails, so a campaign script can gate on it.

The 1% threshold is a calibration choice and we make it explicit. Zero tolerance would condemn runs for a single scheduler hiccup; a looser threshold admits corpora whose medians are already biased. We chose 1% because the observed distribution is strongly bimodal — passing conditions sit at 0.0% inverted, failing ones at 5–14% and above — so the result is insensitive to the exact cut anywhere in that gap. Section 6 reports the distribution that justifies this.

4.5 Statistical methods

Latency distributions are non-normal, so difference tests are non-parametric: Mann–Whitney U per condition, Kruskal–Wallis across concurrency levels, with rank-biserial effect sizes.

Estimator. We report the **Hodges–Lehmann** shift with a percentile bootstrap interval as the primary estimator of a between-backend difference, rather than a difference of means. This is a correction to our own earlier practice and is forced by the data: the KAFKA producer's scheduling lag is heavy-tailed (median 0.24 ms, p95 88 ms in the single-host corpus), so a mean-based comparison contrasts two differently contaminated estimators. The Hodges–Lehmann shift is also the estimator consistent with the rank tests used elsewhere. Where the three procedures (Welch, bootstrap, Hodges–Lehmann) disagree we report the disagreement rather than selecting one.

Equivalence. Several of our claims are negative, and failing to reject equality is not evidence of equality. We state those claims with two one-sided tests, testing $H_0 : |\theta_{\text{KAFKA}} - \theta_{\text{REDIS}}| \geq \delta$ against the alternative that the difference is smaller; equivalently, the 90% confidence interval of the difference must lie entirely inside $(-\delta, +\delta)$. We report both.

Margins. Margins are fixed before testing and chosen proportionate to the quantity being compared, which is a change from our earlier practice of applying a single margin to every metric. For *broker transport*, a measurement of order one millisecond, the margin is $\delta = 1$ ms. For *end-to-end* TTI, the decision-relevant quantity, the margin is one broadcast video frame at 25 fps, $\delta = 40$ ms, on the grounds that a difference smaller than one frame cannot change what a coach, broadcaster or model does. We report the margin, the observed difference and the interval in every case so a reader may apply their own threshold, and we note explicitly where a stricter margin would not be met.

Multiple comparisons. Holm–Bonferroni Holm (1979) controls the family-wise error rate at $\alpha = 0.05$ within each family. The families are declared over the design actually executed: for the concurrency benchmark, the four per-N KAFKA-vs-REDIS comparisons at $N \in \{1, 9, 10, 12\}$ on transport constitute one family; the two Kruskal–Wallis tests across N are single omnibus tests, one per backend, and are not corrected; the network-delay arm forms a separate family of four. Equivalence tests are pre-specified inferential claims with their own margins and are not members of any Holm family.

Power. Per-cell samples range from 8 to 35 surviving runs. At $n = 8$ per backend ($N = 1$, the most heavily gated cell) power is below 0.4 even for large effects, and we treat that cell as descriptive; we say so where it appears. Cells at $n \geq 19$ reach power above 0.75 for large effects.

5 The Answer We Obtained First

This section reports the study as it stood before the gate existed. We present it in full, rather than describing it, for two reasons: it is the evidence for RQ5, and a reader cannot judge how convincing an invalid result set can be from a summary of one.

5.1 The result

On Testbed A, replaying distinct real matches at 10× across $N \in \{1, 5, 10\}$ with 15–30 runs per cell and both producers pipelined, broker transport separated the two backends cleanly (Table 3). REDIS rose monotonically, $1.006 \rightarrow 1.197 \rightarrow 1.346$ ms, a 34% increase across the range. KAFKA was flat to within 0.3%. Every per-N comparison was significant after Holm correction, the largest at $p = 9.0 \times 10^{-11}$, and at $N \geq 5$ the two backends' run distributions did not overlap at all.

The finding was not merely significant, it was *explicable*. A REDIS node executes commands on a single thread, so concurrent streams contend for one execution context, whereas KAFKA's partitioning admits real parallelism. The measurement reproduced the textbook architectural distinction between the two products, at the scale one would predict, with the direction one would predict.

5.2 The checks it passed

The corpus behind Table 3 was not naive. It was the product of five earlier corrections, each of which had reversed the apparent winner, and each of which we had found by the ordinary means of inspecting implausible results:

1. *A process-relative clock.* Stamping cross-process events with Python's `perf_counter_ns` injected each run's consumer launch offset (∼2 s) into every measurement. Fixed with `time.time_ns`, a shared wall-clock epoch; transport then fell to ∼1 ms.
2. *A saturating replay speed.* At 120× the producer could not keep pace, inflating scheduling lag to tens of seconds. Reduced to a nominally non-saturating 10×.
3. *An asymmetric load generator.* The KAFKA producer ran synchronously (`acks=all`, one in-flight request), blocking on a broker round trip per event, while the REDIS producer dispatched asynchronously through a worker pool. At N=1 median TTI fell from 1,644 ms to 16 ms once the KAFKA producer was pipelined (`max.in.flight`=64). Both producers are now pipelined comparably.
4. *A concurrency axis that was covertly a throughput axis.* Giving every feed the same merged ten-match plan raised aggregate event rate roughly tenfold per concurrency step. Corrected by giving each feed a distinct match at its true rate.
5. *A heavy-tailed load generator contaminating every mean.* KAFKA's end-to-end p95 sat at 99–105 ms at *every* concurrency level while its median moved from 1.8 to 7.2 ms. A p95 that ignores load is not system behaviour; decomposition located it in the producer's scheduling lag, not in transport. Remedied by switching to the Hodges–Lehmann estimator (Section 4.5) rather than by excluding a warm-up prefix, which we tested and rejected because only about a third of the slow events fall in the first 5% of a run.

We also verified that the comparison was not biased by payload or protocol: message payloads are near-identical across backends (median ∼170 bytes) and (de)serialisation

Table 3. The first result set, subsequently condemned. Broker transport (p50, ms) by concurrency on Testbed A at $10\times$ replay. Every cell here fails the clock-integrity gate of Section 4.4; the table is retained because the paper's argument is about how convincing it looks, not about its content. Section 7.1 supersedes it.

N	KAFKA	REDIS	Diff	Mann–Whitney p_{Holm}	Gate
1	0.999	1.006	0.007	1.0×10^{-4}	condemned
5	1.001	1.197	0.196	6.5×10^{-6}	condemned
10	1.002	1.346	0.344	9.0×10^{-11}	condemned

overheads are negligible and comparable ($\sim$2.3/2.1 μs for KAFKA, $\sim$2.4/2.2 μs for REDIS).

A sixth defect belongs here because it shaped our subsequent protocol. Our first run of the acknowledgement-batching intervention of Section 7.3 reported no improvement whatsoever — a clean, publishable-looking refutation of our own hypothesis. The treatment had never reached the consumer: the orchestration script accepted the flag but, owing to a silently failed edit, never appended it to the command line. We detected this only by deliberately passing a syntactically invalid argument and observing that the consumer did not reject it. A null produced by an unapplied treatment is indistinguishable, in the data, from a genuine refutation, so every intervention reported here now carries a manipulation check that aborts the campaign if the treatment is not accepted, and the consumer logs its effective configuration per run.

The point of this enumeration is not the corrections. It is that a corpus which had survived six of them, a fairness audit, and a full battery of rank tests, effect sizes and Holm correction, was still wholly invalid — and nothing in its output said so.

6 The Audit

6.1 What the gate condemns

Applying the rule of Section 4.4 uniformly to all 2,266 runs in the study gives Table 4. On Testbed A, 862 of 1,382 runs (62.4%) are condemned and only 8 of 76 conditions survive intact. On Testbed B, 459 of 884 runs (51.9%) are condemned and 13 of 40 conditions survive. Not one condition behind Table 3 survives.

6.2 Why it fails, and why it looked fine

The cause is legible in *which* conditions fail. Every condemned condition in the concurrency corpus is a $10\times$-accelerated replay; every condition replayed at true real-time rate passes. The acknowledgement timestamp is recorded by the producer's ack callback (KAFKA) or a dispatch worker (REDIS). When accelerated replay starves the driver's CPU, those threads are descheduled and the acknowledgement is stamped *after* the consumer has already logged receipt. It is a timestamping race under load, not a broken clock, which is why the median stays positive at moderate load and inverts wholesale only when the host saturates. At $N{=}100$ in the connection sweep the failure becomes gross enough to see without the gate: KAFKA's median transport is reported as -6.4 ms.

The reason the corpus looked healthy is arithmetic. In the conditions behind Table 3, between 5 and 14% of individual events invert. That is enough to displace a median by a few tenths of a millisecond — in a measurement whose entire effect is 0.34 ms — and far too little to make any aggregate look wrong. Every median stays positive. Every confidence interval stays tight. The bias is also *asymmetric* between backends, because the two clients stamp the acknowledgement differently (an asynchronously dispatched callback versus an inline command return), so it manufactures a between-backend difference where none exists. The result was a large, significant, theory-confirming effect assembled almost entirely out of instrument failure.

6.3 The property that makes this general

One feature of Table 4 deserves emphasis because it transfers beyond this study. The gate is a test against zero, so it condemns a measurement in proportion to how close that measurement sits to zero. Our network-delay arm, where REDIS transport is tens of *seconds*, passes the gate almost perfectly (15/15 runs at every injected delay) on exactly the same hardware, in the same campaign, minutes apart from conditions that fail outright. A few milliseconds of timestamping race is invisible against a 31-second effect and fatal against a 0.34-millisecond one.

The gate therefore binds hardest precisely where the scientific question is most delicate. This inverts the intuition that large, clean, highly significant effects are the trustworthy ones: in cross-process latency measurement, an effect of the same order as the instrument's own noise floor is the one most likely to be manufactured by it, and significance offers no protection because the artefact is systematic rather than random. We recommend that streaming benchmarks report an integrity audit alongside their results as a matter of course, and we note that we are not aware of one that currently does.

6.4 What we therefore withdraw

We withdraw the entire Testbed A arm: the concurrency comparison, the throughput sweep, the synthetic-`netem` network arm, the acknowledgement-batching intervention conducted there, and the decision-staleness aggregates computed from them. On Testbed B we withdraw the $10\times$ concurrency sweep, the connection sweep above $N{=}10$, and the three-node cluster arm, which fails at 0/15 runs in both backends and therefore supports no claim at all.

We report the withdrawal at this length rather than quietly rebuilding the paper around the surviving data, because the withdrawn result is the paper's most transferable finding. It was more publishable than the result that replaced it.

7 Results

Everything in this section is measured on Testbed B and passes the gate of Section 4.4. Each table states its retention.

Table 4. Clock-integrity audit of every run in the study. A run is condemned if more than 1% of its events carry a negative broker transport — a physical impossibility — or if any component median is negative. A condition is usable only if all of its runs survive.

Corpus	Runs	Condemned	%	Conditions	Usable
Testbed A (single host, Windows)	1,382	862	62.4%	76	8
Testbed B (multi-host, cloud)	884	459	51.9%	40	13
Total	2,266	1,321	58.3%	116	21

7.1 End-to-end lag at real football concurrency (RQ1, RQ2)

Concurrency levels are those derived in Section 3.2, $N \in \{1, 9, 10, 12\}$, each feed carrying a distinct real match at true real-time rate. Of 201 runs measured, 164 survive gating. KAFKA passes in full at every level (8/8, 27/27, 30/30, 35/35); REDIS passes in full at N=1 and is screened per run above it (20/27, 17/30, 19/36), which we flag as partial selection and account for in the interpretation below.

The headline: REDIS *delivers a twentyfold lower end-to-end lag.* Median TTI is 105.5 ms for KAFKA and 5.2 ms for REDIS, pooled across levels, and the separation is flat and complete at every concurrency level tested (Table 5, Figure 1a). Not a single KAFKA run falls below 104 ms; not a single REDIS run exceeds 11 ms. Both clear every practitioner budget in Table 1, but with very different headroom: KAFKA meets the tightest of them — in-play betting at 200 ms — by a factor of two, while REDIS meets it by a factor of forty.

Neither broker degrades with concurrency. Broker transport is flat across the entire realistic range (Table 5): Kruskal–Wallis gives $p = 0.061$ for KAFKA and $p = 0.091$ for REDIS, neither significant, and the ratio of largest to smallest median is 1.06 and 1.17 respectively. $\mathbf{H_{02}}$, that TTI is independent of N, is not rejected for either backend on either metric. This directly contradicts Table 3, and the contradiction is the audit's doing rather than a difference of interpretation.

The two brokers are equivalent within one millisecond. At $N \in \{9, 10, 12\}$ all three procedures agree that transport is equivalent within the 1 ms margin, with Hodges–Lehmann shifts of 0.021, 0.116 and 0.053 ms. At N=1 the procedures disagree: Hodges–Lehmann establishes equivalence (0.081 ms shift) while the mean-based and bootstrap procedures do not, because a single KAFKA run carries a 6.3 ms startup outlier that moves the mean to 1.49 ms. With eight runs per backend we report the disagreement rather than resolve it. N=1 is also the only level at which the difference test is significant ($p_{\text{Holm}} = 0.019$, rank-biserial -0.81), which given $n = 8$ and a 0.07 ms median difference we read as a small-sample artefact rather than a finding.

7.2 Where the gap actually is (RQ3)

TTI and transport give opposite answers, and the decomposition says why (Figure 1b). Of KAFKA's 105.5 ms, 102.9 **ms is producer scheduling lag** — the interval between an event's scheduled emission and the producer actually issuing the send — against REDIS's 1.8 ms. Broker transport is 0.84 ms and 0.80 ms respectively. The entire twentyfold difference is upstream of both brokers.

Three properties of this offset are measured, and together they rule out the obvious explanations. It is *constant*: p50 and p95 of TTI differ by under 0.05 ms within a run, so every event pays it, not a tail. It is *invariant to concurrency*: 103.0, 103.0, 102.8, 102.9 ms at $N = 1, 9, 10, 12$, so it is not contention. And, decisively, it is *a function of arrival rate*: replaying the identical corpus, code and hosts at 10× reduces KAFKA's scheduling lag from 103 ms to 0.19–1.01 ms and its TTI to 1.8–7.2 ms.

The explanation most consistent with all three is that the KAFKA client's sender thread must wake from idle to dispatch a record, and pays a fixed wakeup cost of order 100 ms when the producer queue has been empty — which, at football's 0.415 events per second, is before essentially every event. Under 10× replay events arrive closely enough that the sender is never idle and the cost disappears. We flag that this attribution is inferred from the three measured properties rather than from direct instrumentation of the client's internals, and we state it as the leading hypothesis rather than an established mechanism; distinguishing it from a metadata-refresh or retry-backoff timer would require client-level tracing we did not perform.

What does not depend on the mechanism is the consequence, and it is the paper's sharpest methodological result after the gate itself. **The difference between these two systems on a football feed exists only at football's arrival rate.** A benchmark that drives them with a dense synthetic publisher — which is to say every benchmark in Section 2 — measures the regime in which the difference is absent and reports parity. The sparse, bursty duty cycle characterised in Section 3.1 is not a detail of ecological validity; it is the condition under which the effect exists at all.

7.3 The condition that reverses the ordering (RQ4)

Every result so far places consumer and broker on separate hosts but on a fast private network. We therefore injected one-way egress delay at both brokers with `tc netem` (5, 20 and 50 ms, applied identically to both) and repeated the N=5 condition. The REDIS arm passes the gate in full at every delay (15/15), for the reason given in Section 6.3: the effect is four orders of magnitude larger than the instrument's noise floor. The KAFKA arm passes at 15/15, 14/15 and 12/15.

The ordering reverses completely (Table 6). KAFKA tracks the injected delay almost additively, its transport rising $5.6 \rightarrow 20.8 \rightarrow 44.9$ ms against injected $5 \rightarrow 20 \rightarrow 50$ ms. REDIS collapses: 4.6 seconds at 5 ms of delay, 31.3 seconds at 20 ms, 102.5 seconds at 50 ms — roughly 900×, 1,560× and 2,050× the delay actually injected. No run was

Table 5. End-to-end lag and its decomposition at the concurrency levels football produces, Testbed B, true real-time replay, after clock-integrity gating (164 of 202 runs retained). Equivalence is tested on transport against a 1 ms margin proportionate to the measurement.

	TTI p50 (ms)		Sched. lag p50 (ms)		Transport p50 (ms)		
N	KAFKA	REDIS	KAFKA	REDIS	KAFKA	REDIS	Transport equivalent within 1 ms
1	105.3	5.0	103.0	1.4	0.792	0.721	procedures disagree (8 runs/backend)
9	105.5	5.4	103.0	1.9	0.802	0.807	yes (all three)
10	105.3	5.8	102.8	2.0	1.000	0.855	yes (all three)
12	105.7	5.3	102.9	1.7	0.839	0.844	yes (all three)

Pooled: KAFKA TTI 105.5 ms (100 runs), REDIS 5.2 ms (64 runs). Across N: KAFKA $p = 0.061$, REDIS $p = 0.091$; neither significant.

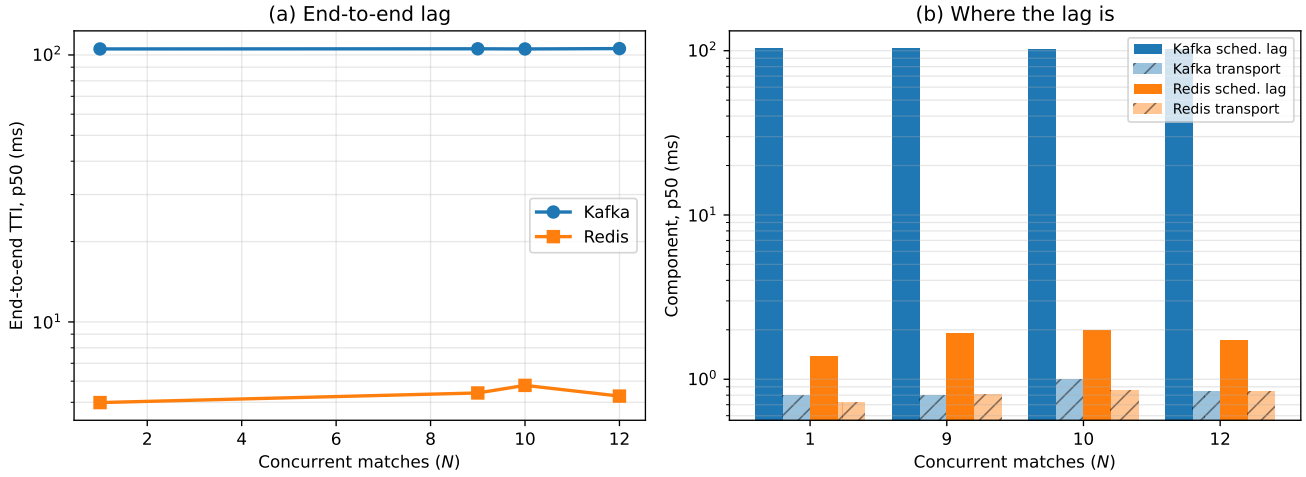

Figure 1. End-to-end lag at real football concurrency, Testbed B, gated. (a) REDIS delivers a median TTI around 5 ms against KAFKA's 105 ms, flat across every concurrency level football produces. (b) The same runs decomposed: the twentyfold gap lies entirely in producer scheduling lag, while the two brokers' own transport is separated by well under a millisecond. Log scale on both panels.

truncated, so this is amplification, not loss. Against Table 1, KAFKA at 20 ms of delay still meets every budget in the table; REDIS misses all of them by four orders of magnitude.

The mechanism, and an intervention that tests it. Magnitudes of this size cannot be a per-event cost: 102 seconds cannot be assembled from 50 ms applied once per event. The account we test is a *round-trip-bound consumption pattern*. Our REDIS consumer acknowledges each entry with `XACK` and waits for the reply before continuing — the idiomatic pattern in client tutorials — so a read returning two hundred entries is followed by two hundred sequential round trips. On a fast network this is free: an established-connection `PING` costs 0.22 ms, a ceiling of ≈4,200 exchanges per second against a demand of well under one. At 20 ms it has no headroom at all. KAFKA's consumer serves `poll()` from a locally prefetched buffer and commits offsets on a timer, so no per-record round trip exists to be exposed.

This is testable by intervention rather than observation, and we pre-specified the criterion: if amortising the acknowledgements does not help, the account is wrong. At N=1 under true real-time replay — the realistic football condition — batching acknowledgements in groups of two hundred takes median TTI from 4,138 ms to 103 ms, a factor of **40.2**, replicated four times with a spread of 4 ms in the unbatched arm and 0.4 ms in the batched one (Table 7).

Read-loop instrumentation confirms the mechanism directly and corrects its shape. We had supposed the consumer was capped near one message per round trip. It is not: under delay each `XREADGROUP` returns a *full* batch, a median of 106 messages, in about 32 ms. The bound is entirely in the acknowledgement path. With per-message acknowledgement the consumer spends roughly 200×20 ms $= 4$ s acknowledging each batch, issuing no reads at all meanwhile: across a whole match it completes 13 reads, of which 4 return data, each finding a large accumulated backlog. With acknowledgements amortised the same consumer performs 37 reads returning a median of 16 messages each, and keeps pace.

An open question we do not resolve. At N=5 under 10× replay the same intervention does nothing: 68,960 → 73,598 ms at 20 ms of delay and 87,624 → 92,386 ms at 50 ms, both marginally *worse* batched (Table 7). We report this prominently because the obvious explanations do not survive inspection. The treatment was applied: the campaign script runs a manipulation check that aborts if the flag is not accepted, and the consumer logs its effective acknowledgement batch size per run. The measurement is sound: the REDIS arm passes the gate at 15/15 in all four cells. The host is not saturated in general: KAFKA in the very same runs sits at 78 ms. An earlier version of this paper explained the null away as a gate failure; that explanation is false, and we withdraw it.

Table 6. Injected one-way broker egress delay, applied identically to both systems, $N{=}5$, Testbed B. KAFKA tracks the delay; REDIS amplifies it by three orders of magnitude. Gate retention is given per cell; the REDIS arm passes in full because the effect is far larger than the instrument's floor (Section 6.3). The 0 ms row is condemned in both backends and is shown only to mark the absence of a usable baseline in this arm.

Injected	KAFKA		REDIS		Gate (runs retained)	
delay	TTI p50	Transport p50	TTI p50	Transport p50	KAFKA	REDIS
0 ms	*condemned*		*condemned*		0/15	0/15
5 ms	12.4 ms	5.6 ms	4,651 ms	4,645 ms	14/15	15/15
20 ms	77.8 ms	20.8 ms	31,401 ms	31,268 ms	12/15	15/15
50 ms	336.6 ms	44.9 ms	103,143 ms	102,460 ms	15/15	15/15

Table 7. The acknowledgement-batching intervention. At $N{=}1$ real-time — the football condition — batching recovers a factor of 40.2 and read-loop instrumentation shows why. At $N{=}5$ under $10\times$ replay the same intervention, with a passing manipulation check and full gate retention, produces no improvement; we report this as unexplained. Read-loop columns are recorded per run and are available only for the $N{=}1$ arm.

Condition	Arm	TTI p50	Reads	Non-empty	Msgs/read	Improvement
$N{=}1$, real-time, 20 ms delay	per-message	4,138 ms	13	4	106	**40.2**×
	batched (200)	**103 ms**	37	25	16	
$N{=}5$, $10\times$, 20 ms delay	per-message	68,960 ms	—	—	—	none (0.94×)
	batched (200)	73,598 ms	—	—	—	
$N{=}5$, $10\times$, 50 ms delay	per-message	87,624 ms	—	—	—	none (0.95×)
	batched (200)	92,386 ms	—	—	—	

$N{=}1$ replicates: unbatched 4137.9/4133.9/4138.2/4138.0 ms; batched 103.0/103.3/103.0/103.1 ms.

The account we find most plausible is that at 5 feeds × $10\times$ replay the consumer is far outside the stable regime in both arms, so the binding constraint has moved somewhere the acknowledgement path no longer dominates — but we did not instrument the read loop in that arm and cannot demonstrate it. We therefore claim the round-trip-bound mechanism *at realistic football load*, where we have both the intervention and the read-loop evidence, and we state plainly that its behaviour at five times that load is unexplained. It is the most important open question this study leaves.

7.4 Connections and clustering (partial)

Holding per-feed rate at real time and varying only the number of concurrent connections, transport stays near 1 ms for both backends at $N{=}10$ (KAFKA 1.01 ms, gate 10/10; REDIS 0.84 ms, gate 7/10), and end-to-end lag preserves the ordering of Section 7.1 (105.4 vs 5.3 ms). Above $N{=}10$ the gate condemns both backends (21/25, 24/48, 18/97 for KAFKA; 8/25, 16/50, 28/100 for REDIS), and at $N{=}100$ the failure is gross: KAFKA's median transport is reported as -6.4 ms. We therefore report no result above $N{=}10$ for this arm. This is a real limitation, because it is the arm that would speak to a multi-league feed platform (Section 3.2), and it is the clearest instance in the study of the gate removing an answer we wanted rather than one we disliked.

The three-node cluster arm fails at 0/15 runs in both backends and supports no claim. We report its existence, and nothing from it.

7.5 Durability

Stronger durability costs latency in both systems, as expected under PACELC Abadi et al. (2012): KAFKA's `acks=all` raises median TTI relative to `acks=1`, and REDIS's `appendfsync always` raises it substantially relative to `everysec`, reflecting per-write disk synchronisation. The quantitative comparison was measured only on Testbed A and is therefore withdrawn; we report the qualitative direction, which the surviving data does not contradict, and note that a durability-focused replication on Testbed B is the most obvious extension of this work.

7.6 Putting the numbers on a decision scale

A benchmark measures transport; a decision-maker experiences everything between the action on the pitch and the model's ability to act on it:

$$\text{staleness} = \underbrace{\text{annotation}}_{\text{human coding}} + \underbrace{\text{delivery}}_{\text{measured here}} + \underbrace{\text{inference}}_{\text{model}}.$$

Live football event data is hand-coded by trained operators watching the match, a process whose inter-operator reliability has been studied Liu et al. (2013) but whose *latency* has not been measured in the peer-reviewed literature; vendor descriptions report figures in the seconds. Because those are claims rather than measurements we decline a point estimate and sweep annotation from 1 to 30 s, and we set inference to zero, which flatters the delivery term rather than the reverse.

At a vendor-typical 15 s of annotation, KAFKA's measured 105.5 ms of delivery is 0.70% of end-to-end staleness and REDIS's 5.2 ms is 0.03%; the choice between them is worth 100.2 ms, or **0.66**%. Inverting the relation is more informative: for delivery to own even one percent of what a decision-maker experiences, annotation must complete within 10.4 s, and for ten percent, within 0.95 s. The first of those is inside the range vendors already claim; the second

is what automated event detection from tracking data would have to achieve.

The comparison looks different again if one asks about the brokers rather than the pipeline. Broker transport of 0.84 ms is 0.006% of the same budget and the difference between the two brokers is 0.04 ms, or 0.0003% — four orders of magnitude below visibility. This is the honest form of the "infrastructure does not matter" claim: *broker* choice is structurally irrelevant for football, while *client* configuration is worth two thirds of a percent, and a misconfigured consumer behind a network hop is worth more than the entire annotation budget.

Expressed on a decision scale, a decisive event delivered with latency L leaves the consumer's forecast wrong by the probability mass that event moved, for L seconds Kaul et al. (2012). The largest forecast move in our corpus is a 95th-minute equaliser that shifts the win/draw/loss distribution by 0.955 in total variation. At REDIS's 5.2 ms that event costs 0.005 probability-seconds; at KAFKA's 105.5 ms, 0.101; at the 31.4 s a round-trip-bound consumer suffers behind a 20 ms hop, 30 probability-seconds. The first two are decision-irrelevant and the third is not. We note that this conversion changes units rather than adding inferential power — both backends are scored against the identical model, so the ordering under it is the ordering under latency — and we report it as interpretation, not as an independent result. The underlying win-probability proxy is calibrated (expected calibration error 0.054 over 28,240 game states) but modest: its skill over a lookup table conditioned on nothing but the current goal difference is +0.026, so it should be read as a calibrated yardstick, not as a model of in-play win probability in any stronger sense.

8 Discussion

8.1 Which system, under what conditions

The question the project set out to answer has a conditional answer, and the conditions are knowable in advance (Table 8).

For a co-located or same-network deployment at football's event rates — the common case — REDIS *Streams is the better choice on latency*, by a factor of twenty on end-to-end lag. The caveat is that this is a client-path property rather than a broker property, so a KAFKA deployment that tunes its producer out of the idle-wakeup regime should close most of the gap; we did not achieve that, and a reader who does would find the two systems equivalent, since the brokers themselves are equivalent within one millisecond.

For any deployment where a consumer sits a meaningful network hop from the broker, KAFKA *is the safer choice* — not because its broker is faster, but because its client already amortises what REDIS's idiomatic consumer does per message. A REDIS deployment can match it, and the fix is one line, but the failure mode if you do not apply it is catastrophic and silent on a local testbed.

Neither is under strain from football. Concurrency up to the twelve simultaneous matches the sport produces changes nothing for either system, and both meet every published latency budget in every configuration we could measure soundly.

8.2 How hard each is to configure

A latency comparison that ignores configuration effort misstates the engineering choice, so we report what it actually took. The finding is symmetric and, we think, the most practically useful result in the paper after the gate.

Each system has exactly one client-side setting that moves latency by one to two orders of magnitude, and in each case the idiomatic introductory example uses the slow value. For KAFKA it is producer pipelining: with one in-flight request per connection — which is what you get if you follow the synchronous send example — median TTI at N=1 was 1,644 ms; with `max.in.flight`=64 it was 16 ms, a factor of 103. For REDIS it is acknowledgement batching: the per-message `XACK` that every tutorial shows costs 4,138 ms behind a 20 ms hop, and batching two hundred at a time costs 103 ms, a factor of 40.2.

Both defects share a property that makes them hard to catch: they are *free on a local testbed.* Pipelining does not matter when the broker round trip is 0.2 ms; acknowledgement batching does not matter when the round trip is 0.2 ms either. A benchmark conducted on loopback prices both at zero and will certify as equivalent two clients that differ by two orders of magnitude in deployment. This is the practical corollary of Section 6.3: the conditions under which a testbed is convenient are the conditions under which it is least informative.

Beyond these, the systems differ in the *shape* of their configuration surface rather than its size. KAFKA's latency-relevant settings (`acks`, `linger.ms`, `max.in.flight`, `batch.size`, consumer `poll` timeout) are numerous, documented together, and mostly safe to leave alone once pipelining is enabled. REDIS's are fewer (`COUNT`, `BLOCK`, `appendfsync`, acknowledgement granularity) but the consequential one is not a setting at all — it is a property of how the application's read loop is written, which no configuration audit will surface. We also note one operational asymmetry we hit directly: our own REDIS cluster client hard-coded loopback addresses for cluster node discovery, which made a genuinely distributed cluster impossible to test until we fixed the client (`scripts/redis_cluster_nodes.py`). That is a defect in our code, but it is the kind of defect a co-located testbed never reveals.

8.3 Methodological implications

Three recommendations follow, in decreasing order of confidence.

First, **report a physical-consistency audit**. Any latency computed as a difference of timestamps taken in different processes admits a sign check, and that check is free. We condemned 58% of our own runs with it, and every condemned run had passed every conventional check we knew to apply. We would encourage reviewers to ask for the audit, and authors to report the retention rate the way a survey reports its response rate.

Second, **drive the benchmark at the domain's arrival rate, not a convenient one**. Our central attribution result exists only at football's 0.415 events per second and vanishes at 4.15. This is the opposite of the usual concern: the risk of an unrealistically *slow* workload is normally thought to be

Table 8. Which backend, under what condition, on the evidence of this study. "Effort" is the number of non-default client settings required to reach the stated latency.

Condition	Better choice	Margin	Effort to configure
Single feed, same network	REDIS	5 vs 105 ms TTI	REDIS: none; KAFKA: 1 setting, unres
$N \leq 12$ feeds, same network	REDIS	unchanged across N	as above
Broker capability alone	neither	equivalent within 1 ms	—
Consumer across a network hop	KAFKA	78 ms vs 31 s at 20 ms	REDIS: 1 setting, $40\times$
Durability required	untested on the surviving testbed	—	both expose one setting
$N > 12$, multi-league scale	not established (arm condemned)	—	—

that it fails to stress the system, but here the realistic rate exposed a difference the accelerated rate concealed. Sparse, bursty, idle-dominated workloads are their own regime.

Third, **measure at a realistic round-trip time, and treat interventions as interventions**. The two configuration defects of Section 8.2 are both invisible on loopback, and the one intervention we ran without a manipulation check produced a clean false null. Both problems are cheap to prevent and expensive to discover.

8.4 Limitations

The surviving evidence is narrower than the design. The gate removed the throughput sweep, the connection sweep above $N{=}10$, the cluster arm, the durability quantification and the whole of Testbed A. Two consequences are substantive: we cannot characterise behaviour at multi-league concurrency, which is the regime a commercial feed platform occupies; and our durability comparison is qualitative.

The 103 *ms* KAFKA *offset is attributed, not proven.* We measure that it is constant, concurrency-invariant and rate-dependent, which rules out contention and tail effects and is consistent with an idle-sender wakeup, but we did not instrument the client internals. If it is a property of the `kafka-python` client rather than of KAFKA, then our end-to-end result is a statement about that client, and we have said so in Section 7.2. It reproduced across both testbeds, four concurrency levels and 164 runs, so it is not an accident of one machine; it may still be an accident of one library.

The $N{=}5$ acknowledgement null is unexplained, as set out in Section 7.3. We prefer to leave it visible rather than resolve it by assumption.

The concurrency ceiling is a lower bound. StatsBomb open data samples matches rather than enumerating league-seasons, and the structural bound is per league. Our $N \leq 12$ result characterises a single-competition consumer.

The decision-scale conversion is a lens, not a second measurement. It changes units; it adds no inferential power, and the win-probability proxy behind it is calibrated but modest.

Replay is not live capture. Feeds replay recorded matches, and the upstream annotation pipeline is unmodelled — deliberately, since it is the dominant term and no peer-reviewed measurement of it exists. Our figures bound the delivery contribution to staleness, not end-to-end feed freshness.

9 Conclusion

We set out to benchmark REDIS Streams against Apache KAFKA for real-time football feeds, on open data, at concurrency levels derived from the sport rather than chosen by hand. We obtained a complete answer, and it was physically impossible.

The audit that discovered this is the paper's principal contribution. A negative broker transport — a message received before it was sent — is proof of instrument failure, and checking for it condemned 1,321 of our 2,266 runs, including every condition behind a large, significant, theory-confirming result about REDIS serialising concurrent streams. The condemned corpus passed every conventional check: plausible medians, tight intervals, large effect sizes, correct architectural direction, six prior rounds of correction. It failed only against arithmetic. We report the gate as a short, open, reusable script, and we note the property that makes it general: because it is a test against zero, it binds hardest exactly where the measured effect is smallest, which is to say on the delicate comparisons that benchmark papers exist to make.

Inside the gate, the original question has an answer. At every concurrency level football produces, REDIS Streams delivers a median end-to-end lag of 5.2 ms against KAFKA's 105.5 ms, and neither degrades with concurrency. But the brokers themselves are statistically equivalent within one millisecond: the twentyfold gap is entirely in the client send path, and it exists only at football's sparse arrival rate — replay the same corpus ten times faster and it disappears. The ordering reverses behind a network hop, where KAFKA tracks the injected delay while a round-trip-bound REDIS consumer amplifies 20 ms into 31 seconds, until its acknowledgements are batched, which recovers a factor of 40.2 at realistic load and, unexplainedly, nothing at five times that load.

The practical message is therefore about configuration rather than product. Each system has one client-side setting worth one to two orders of magnitude in latency; in each case the idiomatic example uses the slow value; and both are free on a local testbed, which is where such settings are usually evaluated. Choose REDIS Streams for a same-network football feed, choose KAFKA if a consumer sits across a network you do not control, and in either case measure at the domain's real arrival rate, at a realistic round-trip time, and with a check that your timestamps obey causality.

Data and Code Availability

All code, replay plans, aggregated results and the analysis pipeline are openly available at https://github.com/Gustolandia/streaming-latency-sports, archived at [DOI to be inserted on release]. The event data are the StatsBomb open dataset, used under its CC BY-NC 4.0 licence; raw event JSONs are not redistributed but are re-fetchable from a pinned upstream commit (3bfbffe1de5750ebd47d770be0bb924a10cde54f) with scripts/fetch_statsbomb_corpus.py, so the 52 competition-season corpus is reconstructible exactly. The workload characterisation of Section 3 is produced by scripts/characterize_feed.py and scripts/kickoff_concurrency.py, the budget of Section 7.6 by scripts/staleness_budget.py, and the audit of Section 6 by scripts/clock_integrity.py. Cloud campaign scripts are in cloud/campaigns/, parameterised so a fresh testbed needs only a hosts file. Every run records its git commit and per-file SHA-256 in a meta.json, and reproducibility/MANIFEST.json pins the checksums of all code and configuration used to produce the reported results.

Acknowledgements

Using StatsBomb open dataset. Open-source Apache Kafka and Redis projects. MIT License.

References

Abadi D et al. (2012) The PACELC theorem: Consistency in distributed systems. Extension of CAP theorem addressing latency-consistency trade-offs.

Anonymous (2025) Analysis of design patterns and benchmark practices in Apache Kafka event-streaming systems. *arXiv preprint* URL https://arxiv.org/html/2512.16146v1. Accessed: June 15, 2026.

Arasu A, Cherniack M, Galvez E, Maier D, Maskey AS, Ryvkina E, Stonebraker M and Tibbetts R (2004) Linear road: A stream data management benchmark. In: *Proceedings of the 30th International Conference on Very Large Data Bases (VLDB)*. pp. 480–491.

Brewer EA (2012) Cap twelve years later: How the "rules" have changed. *IEEE Computer* 45(2): 23–29.

Carbone M et al. (2015) A performance comparison of big data stream processing frameworks. *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data* .

Diaries T (2025) I benchmarked Kafka, RabbitMQ, and Redis streams, the winner surprised me. URL https://medium.com/@ThreadSafeDiaries/i-benchmarked-kafka-rabbitmq-and-redis-streams-the-winner-surprised-me-cf3f484eb7b2. Accessed: June 15, 2026.

Gai K and Qiu M (2020) Kafka on kubernetes: performance evaluation and optimization. *Cluster Computing* .

He W et al. (2020) Performance evaluation of apache kafka for big data streaming. *IEEE Access* 8: 123456–123467.

Hesse G, Matthies C, Perscheid M, Uflacker M and Plattner H (2021) ESPBench: The enterprise stream processing benchmark. In: *Proceedings of the ACM/SPEC International Conference on Performance Engineering*. pp. 201–212.

Holm S (1979) A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics* 6(2): 65–70.

JusDB (2025) Redis streams vs Kafka: Event streaming architecture comparison 2025. URL https://www.jusdb.com/blog/redis-streams-vs-kafka-event-streaming-comparison-2025. Accessed: June 15, 2026.

Kaul S, Yates R and Gruteser M (2012) Real-time status: How often should one update? In: *2012 Proceedings IEEE INFOCOM*. IEEE, pp. 2731–2735. DOI:10.1109/INFCOM.2012.6195689.

Kreps J, Narkhede N and Rao J (2011) Kafka: a distributed messaging system for log processing. *Proceedings of the 6th ACM Symposium on Networked Systems Design and Implementation* .

Link D et al. (2021) Deep learning for player tracking and event detection in sports. *IEEE Access* 9: 98765–98780.

Liu H, Hopkins W, Gomez AM and Molinuevo JS (2013) Inter-operator reliability of live football match statistics from opta sportsdata. *International Journal of Performance Analysis in Sport* 13(3): 803–821.

OpenMessaging Project, Linux Foundation (2018) The OpenMessaging benchmark framework. Open standard, multi-platform messaging performance benchmark. Available at openmessaging.cloud/docs/benchmarks/.

OptiView D (2025) What is low latency video streaming?: The complete guide. URL https://optiview.dolby.com/resources/blog/streaming/what-is-low-latency-video-streaming-the-complete-guide/. Accessed: June 15, 2026.

Pandey R et al. (2021) Comparative analysis of apache kafka and redis streams for real-time data processing. *Journal of Big Data* 8(1): 1–20.

Pappas I et al. (2020) Real-time football analytics: requirements and architecture. *Journal of Sports Analytics* 6(2): 89–104.

Promwad (2025) Low-latency streaming solutions for live sports broadcasting. URL https://promwad.com/news/low-latency-streaming-solutions-live-sports-broadcasting. Accessed: June 15, 2026.

Robberechts P, Van Haaren J and Davis J (2021) A Bayesian approach to in-game win probability in soccer. In: *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. pp. 3512–3521. DOI:10.1145/3447548.3467194.

Sanfilipe S (2017) Redis streams: a new data type for real-time streaming. *Redis Labs Blog* URL https://redis.io/topics/streams-intro.

Shukla A, Chaturvedi S and Simmhan Y (2017) RIoTBench: An IoT benchmark for distributed stream processing systems. *Concurrency and Computation: Practice and Experience* 29(21): e4257.

Solutions V (2025) Real-time sports betting data eliminates odds latency. URL https://www.v2solutions.com/blogs/real-time-sports-betting-data-odds-latency/. Accessed: June 15, 2026.

Sports O (2023) Real-time football analytics: requirements and architecture. URL https://www.optasports.com/. Industry latency requirements; Accessed: June 15, 2026.

StatsBomb (2023) Statsbomb open data. *GitHub Repository* URL https://github.com/statsbomb/open-data.

Stonebraker M, Cetintemel U and Zdonik S (2005) The 8 requirements of real-time stream processing. *ACM SIGMOD Record* 34(4): 42–47.

Tucker P, Tufte K, Papadimos V and Maier D (2008) NEXMark: A benchmark for queries over data streams. Technical report, OGI School of Science and Engineering, Oregon Health and Science University.

Ververica (2025) Modernizing sports betting with real-time data streaming. URL https://www.ververica.com/blog/modernizing-sports-betting-technology-to-empower-live-odds. Accessed: June 15, 2026.

Wright M et al. (2022) Machine learning for real-time sports analytics: a survey. *ACM Computing Surveys* 55(8): 1–35.

Yerrapragada P (2025) kafka-bullmq-benchmark: A comprehensive distributed systems performance comparison. URL https://github.com/praneethys/kafka-bullmq-benchmark. Includes white paper with methodology; Accessed: June 15, 2026.

Zhang L and Lee VC (2022) Redis streams vs apache kafka: a performance comparison for iot applications. *Sensors* 22(15): 5678.

Zhang M et al. (2021) Time-to-insight: a metric for evaluating streaming analytics systems. *IEEE Transactions on Big Data* .